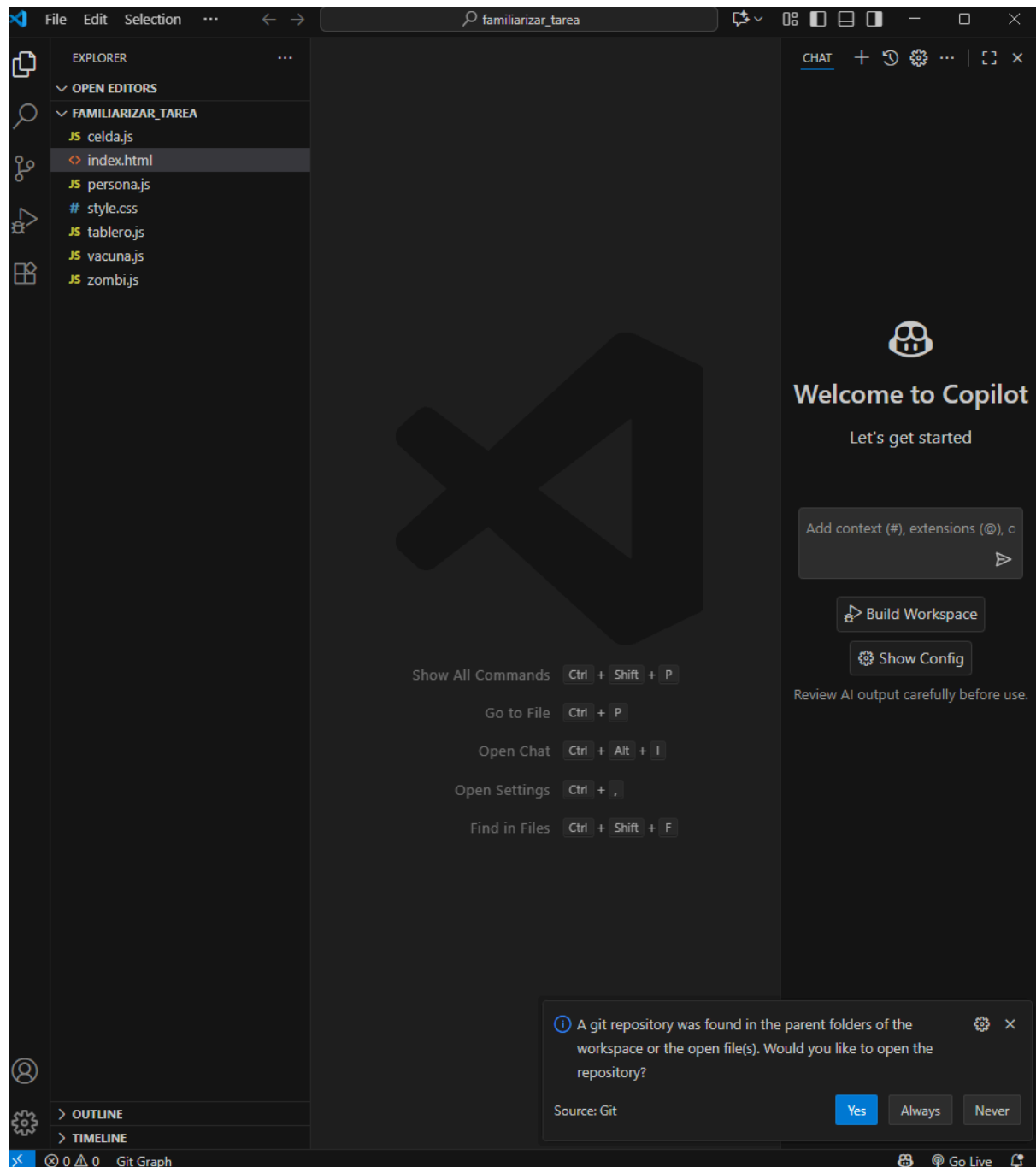


A6.1. UT2 Práctica Guiada 1: IDEs

Actividad 1



Actividad 2

¿En que lenguaje de programación está escrito?

Python

¿Es un lenguaje compilado o interpretado?

Interpretado

¿Has podido ejecutarlo directamente nada más instalar el Visual Studio Code?

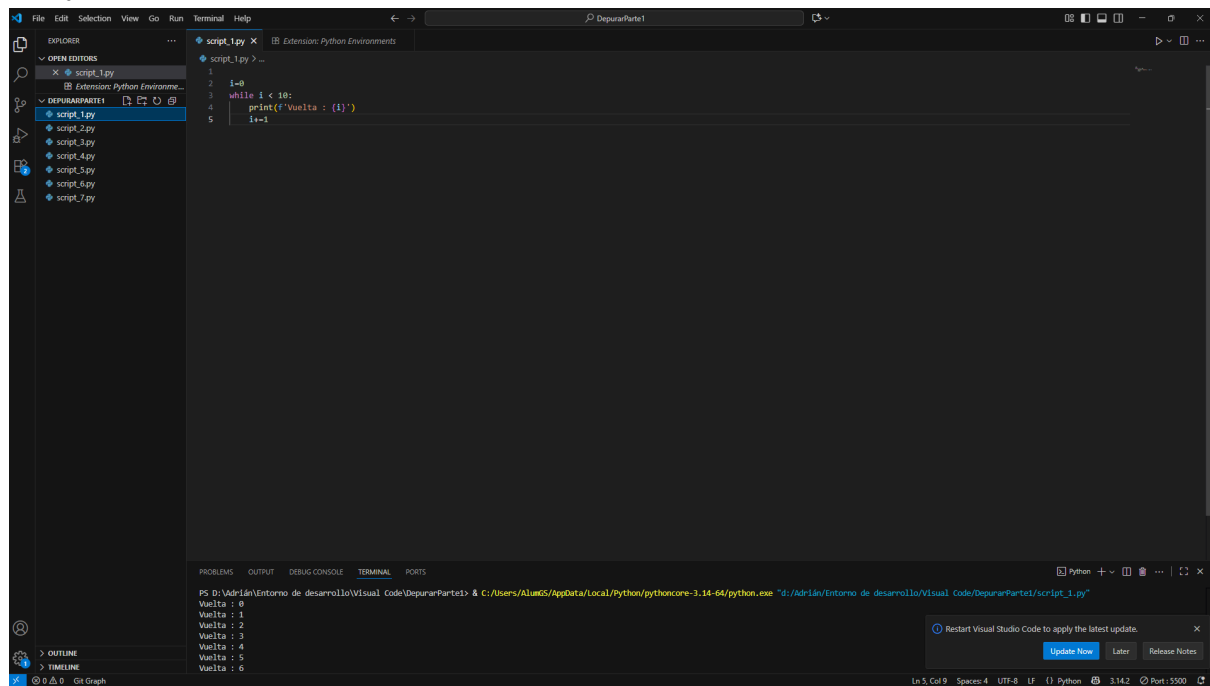
No

¿Que tienes que instalar para ejecutarlo?

Python

¿Has tenido que instalar algún plugin adicional? ¿Cual?

Sí, python



Actividad 3

Información

Python es un lenguaje de programación de alto nivel, interpretado y de propósito general, creado para ser fácil de leer y escribir, con una sintaxis clara e intuitiva. Es multiplataforma (funciona en Windows, Linux y macOS) y soporta múltiples estilos de programación como imperativo, estructurado, orientado a objetos y funcional.

Principales características

- Alto nivel: permite programar sin tener que gestionar detalles complejos de bajo nivel.
- Sintaxis sencilla y legible: el código se parece al inglés, lo que facilita su aprendizaje y comprensión.
- Interpretado: no necesita compilación previa; el intérprete ejecuta el código directamente.
- Tipado dinámico: no hace falta declarar tipos de variables, lo que agiliza la escritura de código.
- Extensa biblioteca estándar y ecosistema de paquetes: incluye miles de módulos y librerías para muchos usos.
- Multiplataforma: el mismo código puede ejecutarse en distintos sistemas operativos.
- Comunidad muy activa: hay abundantísima documentación, recursos y soporte.

Puntos fuertes

1. Fácil de aprender y usar: ideal para principiantes y para desarrollar rápidamente prototipos o aplicaciones simples.
2. Gran ecosistema de librerías y frameworks: por ejemplo, NumPy y Pandas (ciencia de datos), Django y Flask (web), TensorFlow y PyTorch (IA).
3. Lenguaje versátil: sirve tanto para scripting y automatización como para aplicaciones complejas.
4. Alto nivel de abstracción: permite centrarse en la lógica y no en detalles de memoria o compilación.
5. Amplio uso profesional: utilizado por grandes empresas tecnológicas en proyectos reales.

Puntos débiles

1. Velocidad de ejecución relativamente baja: al ser interpretado y dinámico, suele ser más lento que lenguajes compilados como C/C++ o Java.
2. Consumo de memoria: usa más memoria que lenguajes más bajos, lo que puede ser un problema en sistemas limitados.
3. Limitaciones en multi-hilo: el *Global Interpreter Lock (GIL)* impide usar a la vez múltiples núcleos de CPU eficientemente en programas de hilos.
4. No ideal para desarrollo móvil o de videojuegos de alto rendimiento: hay herramientas, pero no es su fuerte.
5. Errores en tiempo de ejecución: el tipado dinámico puede hacer que algunos errores solo aparezcan cuando el programa está ejecutándose.

Principales campos de aplicación hoy

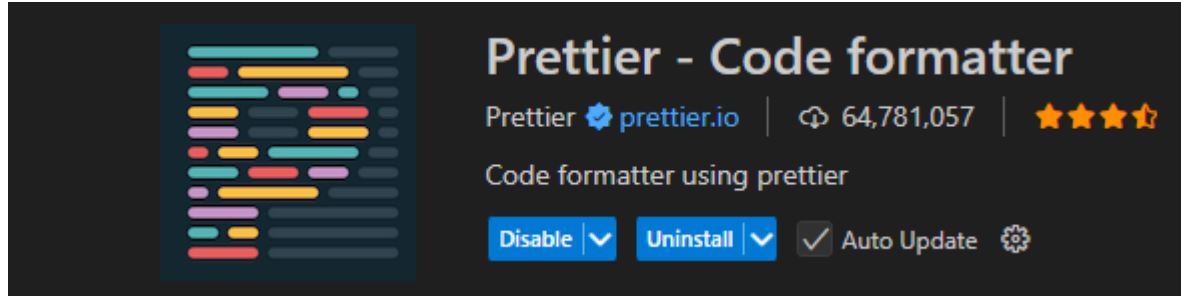
Python se usa ampliamente en muchas áreas de la tecnología:

- Desarrollo web: servidores y API con frameworks como Django o Flask.
- Ciencia de datos y análisis: con librerías como NumPy, Pandas, Matplotlib o SciPy.
- Inteligencia artificial y aprendizaje automático: TensorFlow, PyTorch y Scikit-learn.
- Automatización y scripting: tareas repetitivas, pruebas o procesos automáticos.
- Cálculo científico: simulaciones y cálculos matemáticos complejos.
- Desarrollo de aplicaciones de escritorio: con interfaces gráficas usando Tkinter, PyQt, etc.
- Ciberseguridad y redes: creación de herramientas y scripts de análisis.
- Web scraping y extracción de datos: con librerías como BeautifulSoup o Scrapy.

Plugins

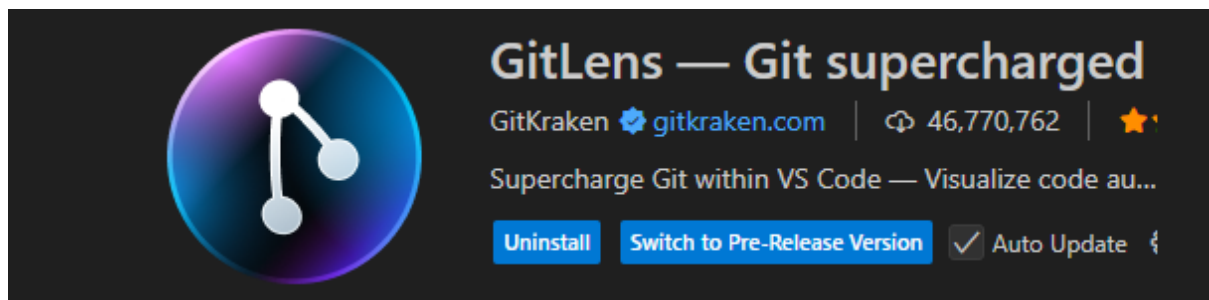
1. Prettier – Code Formatter

Formato automático de código para mantener estilo uniforme en HTML, CSS, JavaScript, Python y otros. Mejora legibilidad y evita errores de estilo al programar.

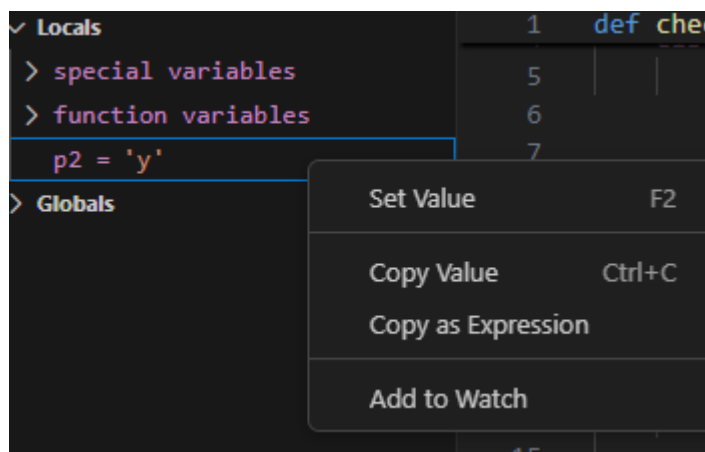
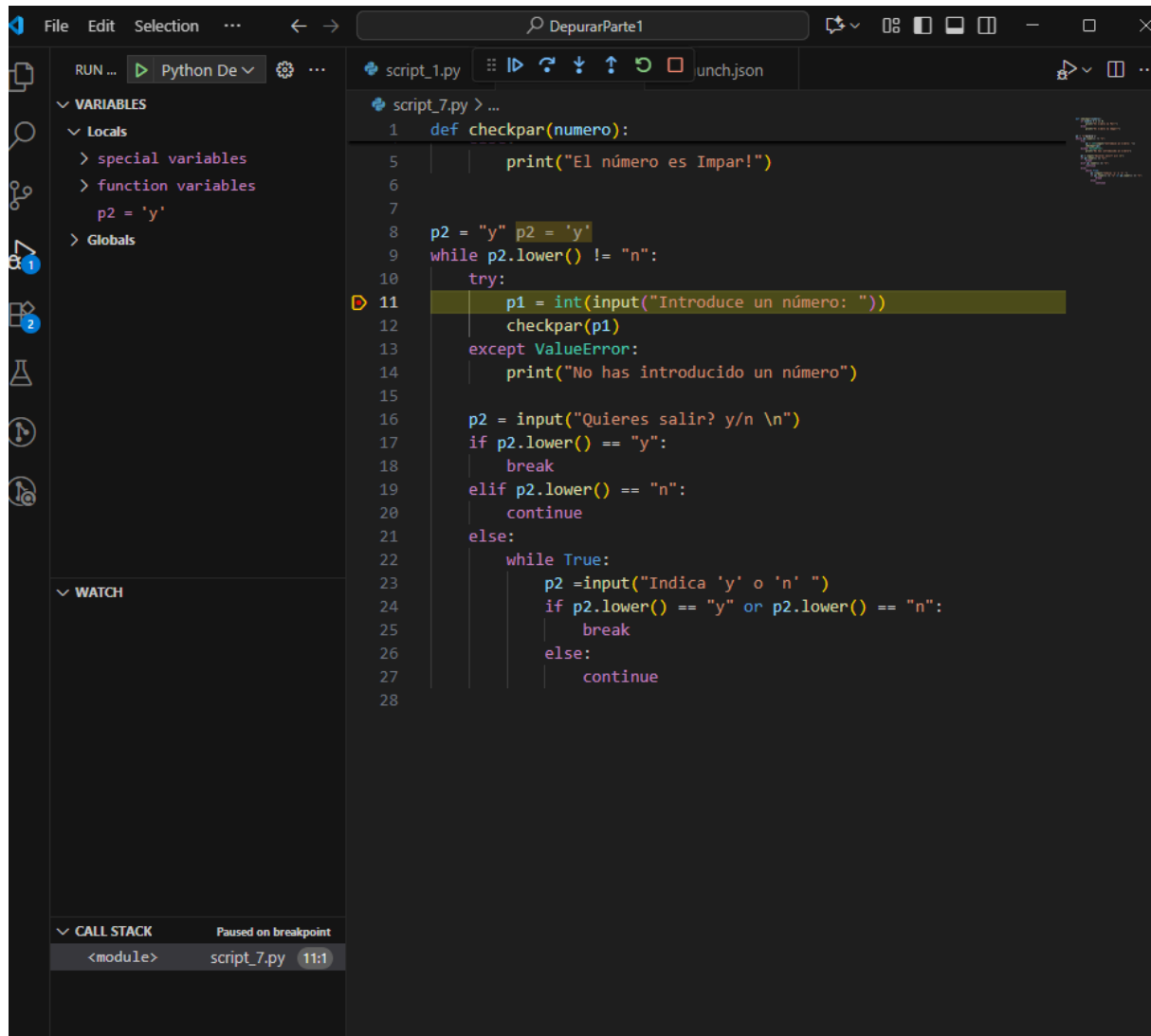


2. GitLens — Git supercharged

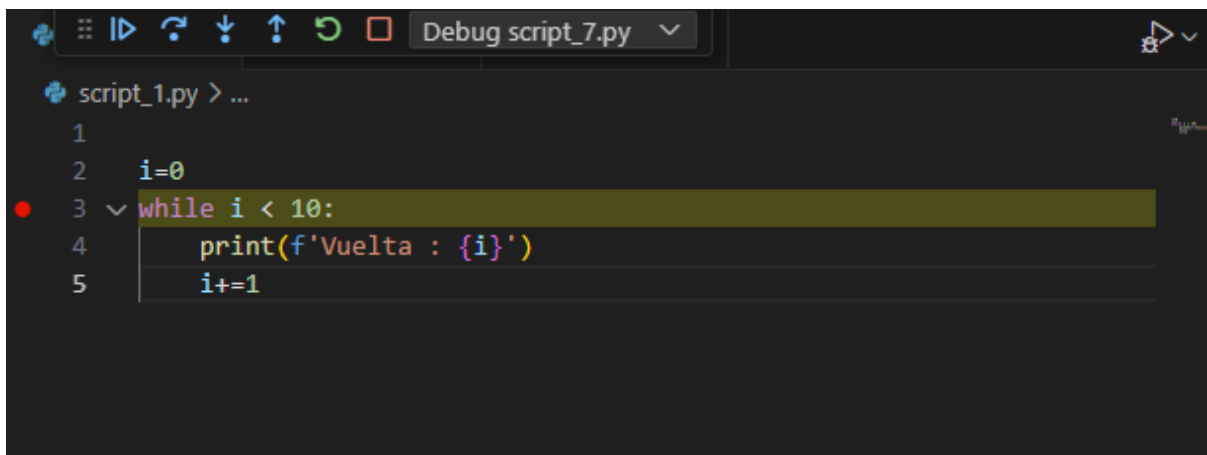
Potencia las capacidades de Git dentro de VS Code mostrando historial de cambios, quién escribió cada línea de código, anotaciones, ramas y más. Muy útil para trabajar en equipo.



Actividad 4



4. 1. Pon un punto de ruptura en el script_1.py



Continue (Continuar)

- Ejecuta el programa hasta el siguiente breakpoint o hasta que termine.
- No entra en funciones ni se detiene línea a línea.

Step Over (Paso a paso sin entrar)

- Ejecuta la línea actual completa.
- Si hay una función, no entra dentro de ella.

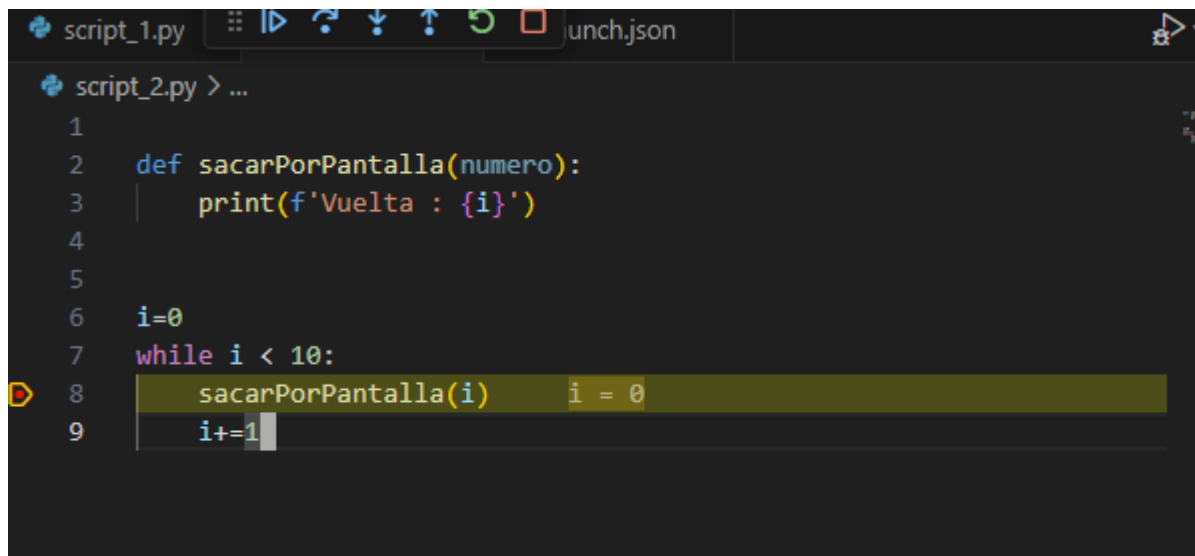
Step Into (Entrar en la función)

- Entra dentro de la función llamada en la línea actual.

Step Out (Salir de la función)

- Termina de ejecutar la función actual y vuelve a la línea que la llamó.

4.2. Ahora ejecuta el script_2.py



```
script_1.py  script_2.py > ...  launch.json
1
2 def sacarPorPantalla(numero):
3     print(f'Vuelta : {i}')
4
5
6 i=0
7 while i < 10:
8     sacarPorPantalla(i)    i = 0
9     i+=1
```

1. Continue (Continuar)

- Salta desde el punto de ruptura hasta el siguiente punto de ruptura o hasta que termine el programa.
- No entra dentro de funciones, solo sigue la ejecución normal.

2. Step Over (Saltar sobre / Siguiente línea)

- Ejecuta la línea actual y se detiene en la siguiente línea del mismo nivel.
- Si la línea llama a una función, ejecuta la función completa sin entrar en ella.

3. Step Into (Entrar en / Ir dentro)

- Si la línea actual llama a una función, entra dentro de esa función y se detiene en su primera línea.
- Permite depurar línea por línea dentro de la función.

4. Step Out (Salir de / Terminar función)

- Si estás dentro de una función, ejecuta todo lo que queda de esa función y se detiene cuando vuelve a la función que la llamó.
- Sirve para salir rápido de funciones que ya no quieres depurar línea por línea.

4.3. script 3

ocurre este error:

Cuando `i = 5`, `colores[5]` **no existe** → `IndexError: list index out of range`

lo arregle poniendo un punto de ruptura en la línea del `print`, le di a `step into`, luego `step out` y por último `continue`.

```
script_3.py > ...
1 colores = ["rojo", "amarillo", "naranja", "azul", "morado"]
2
3 for i in range(len(colores) + 1):
4     print(colores[i]) colores = ['rojo', 'amarillo', 'naranja', 'az
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

Adrián\Entorno de desarrollo\Visual Code\DepurarParte1'; & 'c:\Users\AlumGS\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\AlumGS\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52390' '--' 'd:\Adrián\Entorno de desarrollo\Visual Code\DepurarParte1\script_3.py'

rojo
amarillo

Python
Python Deb...
Python Deb...
Python Deb...

4.4. ¿Se produce algún error?¿Por qué? Explica que está ocurriendo
Sí, esto es debido a que la lista tiene 5 elementos, pero estás iterando hasta 6 y hace un error de índice.
Además, intentas sumar un número a un string, lo que hace error de tipo.

4.5. Ejecuta el script_5.py

Proceso:

Step Into, Step Over, Step Out y Continue

a) Step Into (Entrar en la función)

El depurador entra en la función operaciones línea por línea.

Variables iniciales: $a = 5$, $b = 10$.

Se detiene en `resultado_intermedio = a * 2`.

b) Step Over (Ejecutar línea y pasar a la siguiente)

`resultado_intermedio = a * 2`

Se calcula $5 * 2 = 10$.

Variable `resultado_intermedio` se actualiza a 10.

`resultado_intermedio_2 = resultado_intermedio * 10`

Se calcula $10 * 10 = 100$.

Variable `resultado_intermedio_2` se actualiza a 100.

`resultado_final = resultado_intermedio + b + resultado_intermedio_2`

Se calcula $10 + 10 + 100 = 120$.

Variable `resultado_final` se actualiza a 120.

c) Step Out (Salir de la función)

Ejecuta todo lo que queda de la función y retorna el valor `resultado_final = 120`.

Este valor se asigna a `resultado_operacion`.

d) Continue (Continuar la ejecución hasta el final)

El depurador ejecuta todo lo que queda del programa.

Se llega a `print("Resultado final:", resultado_operacion)` y se imprime:

Resultado final: 120

4.6. Ejecuta el script_6.py.

el script falla con un error de conversión si no se meten números enteros.

4.7. Ejecuta el script_7.py.

Si el usuario introduce letras en lugar de un número, el programa intenta convertir esa entrada a un entero usando `int()`. Como las letras no se pueden convertir a entero, Python lanza un `ValueError`.

Gracias al bloque `try-except` que rodea la conversión, el programa no se detiene; en su lugar, captura el error y muestra el mensaje:

“No has introducido un número”

Después de mostrar este mensaje, el programa continúa su ejecución y pregunta al usuario si desea salir (y/n). De esta manera, el script gestiona entradas no válidas de forma segura, informando al usuario del error sin interrumpir la ejecución.

Al poner a prueba el programa con distintos valores por teclado, se observó lo siguiente:

Número par: al introducir 4, el programa respondió correctamente con El número es Par! y, al indicar n para no salir, continuó pidiendo otro número.

Número impar: al introducir 7, el programa indicó El número es Impar! y, al introducir y para salir, terminó la ejecución como se esperaba.

Entrada no numérica: al introducir letras como "abc", el programa capturó el error mediante el bloque `try-except` y mostró el mensaje No has introducido un número. Después, preguntó si se quería salir, permitiendo continuar la ejecución sin que el programa se detuviera abruptamente.

Uso de mayúsculas en la opción de salida: al introducir Y o N, el programa interpretó correctamente las respuestas gracias al uso de `p2.lower()`, demostrando que es insensible a mayúsculas o minúsculas.

Entrada inválida en la pregunta de salida: al introducir un valor distinto de y o n, el programa entró en el bucle interno, solicitando de nuevo la entrada hasta que se introdujo una respuesta válida.

En general, el programa funciona correctamente, maneja errores de entrada numérica y valida la opción de salida, mostrando un comportamiento robusto y controlado.